



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Approved by AICTE, Affiliated to JNTUH, Hyderabad-500029

Prepared By:

M. Asha Jyothi

Assistant Professor, CSE(AIML)

KMIT, Hyderabad

Exercise: Generating Synthetic Fashion Images Using a GAN

Objective

In this exercise we will understand the working principle of a Generative Adversarial Network (GAN) by developing a Generator and Discriminator and training them against each other to generate synthetic fashion images.

we will also analyse whether the generated images are:

Realistic

Diverse

Similar to the training data

Affected by training instability or mode collapse

Dataset

Fashion-MNIST

Use the Fashion-MNIST dataset available through TensorFlow/Keras.

The dataset contains grayscale images of fashion items such as:

T-shirts/tops Trousers Pullovers Dresses Coats Sandals Shirts Sneakers Bags Ankle boots

Image size:

28 × 28 pixels

For this exercise, you may initially use the complete dataset or a manageable subset depending on your available computing resources.

Part A – Understanding the GAN

Q1. What is the difference between a GAN and an Autoencoder?

Explain specifically:

Input to the model

Architecture

Purpose

Training objective

Type of output produced

Q2. Explain the adversarial relationship.

Complete the following:

Generator tries to: _____

Discriminator tries to: _____

Explain how improvement of one network affects the other.

architecture of GAN.

Random Noise



Generator



Synthetic Image



Discriminator



Probability: Real / Fake

Generator:

Input dimension Number of layers Activation functions Output image size

Discriminator:

Input image size Number of layers Activation functions Output

Part B: Implement the GAN

Using Python and TensorFlow/Keras:

Step 1 – Load the dataset

Load Fashion-MNIST and perform appropriate preprocessing.

Your program should:

Load the images

Normalize pixel values

Reshape the data if required

Select an appropriate training subset if necessary

Step 2 – Build the Generator

Create a Generator that accepts a random latent vector and produces a 28×28 grayscale image.

The Generator should progressively transform:

Random Noise

↓

Dense / Projection Layer

↓

Upsampling / Transposed Convolution

↓

Upsampling / Transposed Convolution

↓

28×28 Generated Image

Step 3 – Build the Discriminator

Create a Discriminator that receives an image and predicts whether it is:

REAL = 1

FAKE = 0

Step 4 – Define the losses

Implement appropriate loss functions for:

Discriminator

Generator

Explain what each loss represents.

Step 5 – Train the GAN

During training, alternate between:

Discriminator training

Train the Discriminator using:

Real Images → Real

Generated Images → Fake

Generator training

Train the Generator so that:

Generated Images → Discriminator predicts Real

Record the Generator and Discriminator losses during training.

```
# =====  
# BASIC GAN - FASHION MNIST  
# Complete Google Colab Implementation  
# =====  
  
# -----  
# 1. IMPORT LIBRARIES  
# -----  
  
import numpy as np  
import matplotlib.pyplot as plt  
import tensorflow as tf  
  
from tensorflow.keras import layers  
from tensorflow.keras.datasets import fashion_mnist  
  
print("TensorFlow Version:", tf.__version__)  
  
# Check GPU  
print("GPU Available:", tf.config.list_physical_devices('GPU'))  
  
TensorFlow Version: 2.20.0  
GPU Available: []  
  
# -----  
# 2. LOAD FASHION-MNIST DATASET  
# -----  
  
(X_train, _), (_, _) = fashion_mnist.load_data()  
  
print("Original Dataset Shape:", X_train.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 _____ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 _____ 0s 0us/step
Original Dataset Shape: (60000, 28, 28)
```

```
# -----
# 3. PREPROCESS THE DATA
# -----
```

```
# Convert pixel values from [0,255] to [-1,1]
X_train = X_train.astype("float32")
X_train = (X_train - 127.5) / 127.5
```

```
# Add channel dimension
# From: (60000, 28, 28)
# To:   (60000, 28, 28, 1)
```

```
X_train = np.expand_dims(X_train, axis=-1)
```

```
print("After preprocessing:", X_train.shape)
print("Minimum value:", X_train.min())
print("Maximum value:", X_train.max())
```

```
After preprocessing: (60000, 28, 28, 1)
Minimum value: -1.0
Maximum value: 1.0
```

```
# -----
# 4. CREATE TENSORFLOW DATASET
# -----
```

```
BUFFER_SIZE = 60000
BATCH_SIZE = 128
```

```
dataset = tf.data.Dataset.from_tensor_slices(X_train)
```

```
dataset = dataset.shuffle(BUFFER_SIZE).batch(
    BATCH_SIZE,
    drop_remainder=True
)
```

```
print("Dataset prepared.")
```

Dataset prepared.

```
# =====  
# 5. BUILD THE GENERATOR  
# =====
```

```
LATENT_DIM = 100
```

```
generator = tf.keras.Sequential([  
    # Input: random noise vector of size 100  
    layers.Input(shape=(LATENT_DIM,)),  
  
    # Convert noise into a larger representation  
    layers.Dense(7 * 7 * 128, use_bias=False),  
    layers.BatchNormalization(),  
    layers.LeakyReLU(),  
  
    # Reshape into feature maps  
    layers.Reshape((7, 7, 128)),  
  
    # 7x7 -> 14x14  
    layers.Conv2DTranspose(  
        64,  
        kernel_size=5,  
        strides=2,  
        padding="same",  
        use_bias=False  
    ),  
    layers.BatchNormalization(),  
    layers.LeakyReLU(),  
  
    # 14x14 -> 28x28  
    layers.Conv2DTranspose(  
        32,  
        kernel_size=5,  
        strides=2,  
        padding="same",  
        use_bias=False  
    ),  
    layers.BatchNormalization(),  
    layers.LeakyReLU(),  
  
    # Final image  
    layers.Conv2DTranspose(  
        1,  
        kernel_size=5,  
        strides=1,
```

```

        padding="same",
        activation="tanh"
    )

```

```

    ])

```

```

generator.summary()

```

```

Model: "sequential"

```

Layer (type) Param #	Output Shape	
dense (Dense) 627,200	(None, 6272)	
batch_normalization (BatchNormalization) 25,088	(None, 6272)	
leaky_re_lu (LeakyReLU) 0	(None, 6272)	
reshape (Reshape) 0	(None, 7, 7, 128)	
conv2d_transpose (Conv2DTranspose) 204,800	(None, 14, 14, 64)	
batch_normalization_1 (BatchNormalization) 256	(None, 14, 14, 64)	
leaky_re_lu_1 (LeakyReLU) 0	(None, 14, 14, 64)	

51,200	conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	
128	batch_normalization_2 (BatchNormalization)	(None, 28, 28, 32)	
0	leaky_re_lu_2 (LeakyReLU)	(None, 28, 28, 32)	
801	conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	

Total params: 909,473 (3.47 MB)

Trainable params: 896,737 (3.42 MB)

Non-trainable params: 12,736 (49.75 KB)

```
# -----
# 6. TEST THE GENERATOR
# -----

random_noise = tf.random.normal([1, LATENT_DIM])
generated_image = generator(random_noise, training=False)
print("Generated image shape:", generated_image.shape)
Generated image shape: (1, 28, 28, 1)

# =====
# 7. BUILD THE DISCRIMINATOR
# =====

discriminator = tf.keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    # 28x28 -> 14x14
```

```

layers.Conv2D(
    64,
    kernel_size=5,
    strides=2,
    padding="same"
),
layers.LeakyReLU(),
layers.Dropout(0.3),

# 14x14 -> 7x7
layers.Conv2D(
    128,
    kernel_size=5,
    strides=2,
    padding="same"
),
layers.LeakyReLU(),
layers.Dropout(0.3),

layers.Flatten(),

# Output: probability that image is real
layers.Dense(1)

```

```

1)

```

```

discriminator.summary()

```

```

# -----
# 8. TEST DISCRIMINATOR
# -----

```

```

decision = discriminator(generated_image)

```

```

print("Discriminator output:", decision.numpy())

```

```

Model: "sequential_1"

```

Layer (type) Param #	Output Shape	
conv2d (Conv2D) 1,664	(None, 14, 14, 64)	
leaky_re_lu_3 (LeakyReLU) 0	(None, 14, 14, 64)	

0	dropout (Dropout)	(None, 14, 14, 64)
204,928	conv2d_1 (Conv2D)	(None, 7, 7, 128)
0	leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)
0	dropout_1 (Dropout)	(None, 7, 7, 128)
0	flatten (Flatten)	(None, 6272)
6,273	dense_1 (Dense)	(None, 1)

Total params: 212,865 (831.50 KB)

Trainable params: 212,865 (831.50 KB)

Non-trainable params: 0 (0.00 B)

Discriminator output: [[-0.00140721]]

```
# =====
# 9. DEFINE LOSS FUNCTION
# =====
```

```
cross_entropy = tf.keras.losses.BinaryCrossentropy(
    from_logits=True
)
```

```
# -----
# Generator Loss
# -----
```

```
def generator_loss(fake_output):
```

```
    # Generator wants discriminator
```

```

    # to classify fake images as REAL (1)

    return cross_entropy(
        tf.ones_like(fake_output),
        fake_output
    )

# -----
# Discriminator Loss
# -----

def discriminator_loss(real_output, fake_output):

    # Real images should be classified as 1
    real_loss = cross_entropy(
        tf.ones_like(real_output),
        real_output
    )

    # Fake images should be classified as 0
    fake_loss = cross_entropy(
        tf.zeros_like(fake_output),
        fake_output
    )

    total_loss = real_loss + fake_loss

    return total_loss

# =====
# 10. OPTIMIZERS
# =====

generator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002,
    beta_1=0.5
)

discriminator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002,
    beta_1=0.5
)

# =====
# 11. FIXED RANDOM NOISE
# =====

# We use the SAME noise every time we visualize results.
# This allows students to observe how the generated images
# improve during training.

```

```

NUM_EXAMPLES = 16

seed = tf.random.normal(
    [NUM_EXAMPLES, LATENT_DIM]
)

# =====
# 12. TRAINING STEP
# =====

@tf.function
def train_step(images):

    # Generate random noise
    noise = tf.random.normal(
        [BATCH_SIZE, LATENT_DIM]
    )

    # -----
    # Train Discriminator
    # -----

    with tf.GradientTape() as disc_tape:

        generated_images = generator(
            noise,
            training=True
        )

        real_output = discriminator(
            images,
            training=True
        )

        fake_output = discriminator(
            generated_images,
            training=True
        )

        disc_loss = discriminator_loss(
            real_output,
            fake_output
        )

    gradients_of_discriminator = disc_tape.gradient(
        disc_loss,
        discriminator.trainable_variables
    )

    discriminator_optimizer.apply_gradients(

```

```

        zip(
            gradients_of_discriminator,
            discriminator.trainable_variables
        )
    )

# -----
# Train Generator
# -----

noise = tf.random.normal(
    [BATCH_SIZE, LATENT_DIM]
)

with tf.GradientTape() as gen_tape:
    generated_images = generator(
        noise,
        training=True
    )

    fake_output = discriminator(
        generated_images,
        training=True
    )

    gen_loss = generator_loss(
        fake_output
    )

gradients_of_generator = gen_tape.gradient(
    gen_loss,
    generator.trainable_variables
)

generator_optimizer.apply_gradients(
    zip(
        gradients_of_generator,
        generator.trainable_variables
    )
)

return gen_loss, disc_loss

```

```

# =====
# 13. FUNCTION TO DISPLAY GENERATED IMAGES
# =====

```

```

def generate_and_plot_images(model, epoch):

```

```

    predictions = model(
        seed,
        training=False
    )

    predictions = (predictions + 1) / 2.0
    fig = plt.figure(figsize=(8, 8))
    for i in range(NUM_EXAMPLES):
        plt.subplot(4, 4, i + 1)

        plt.imshow(
            predictions[i, :, :, 0],
            cmap="gray"
        )

        plt.axis("off")

    plt.suptitle(
        f"Generated Images - Epoch {epoch}",
        fontsize=16
    )

    plt.tight_layout()
    plt.show()

# =====
# 14. TRAIN THE GAN
# =====

EPOCHS = 30

generator_losses = []
discriminator_losses = []

print("\nStarting GAN Training...\n")

for epoch in range(1, EPOCHS + 1):
    gen_loss_total = 0
    disc_loss_total = 0

    num_batches = 0

    for image_batch in dataset:
        gen_loss, disc_loss = train_step(

```

```

        image_batch
    )

    gen_loss_total += gen_loss
    disc_loss_total += disc_loss

    num_batches += 1

# Average losses
    gen_loss_avg = (
        gen_loss_total / num_batches
    )

    disc_loss_avg = (
        disc_loss_total / num_batches
    )

    generator_losses.append(
        float(gen_loss_avg)
    )

    discriminator_losses.append(
        float(disc_loss_avg)
    )

    print(
        f"Epoch {epoch:02d}/{EPOCHS} | "
        f"Generator Loss: {gen_loss_avg:.4f} | "
        f"Discriminator Loss: {disc_loss_avg:.4f}"
    )

# Display generated images every 5 epochs
    if epoch == 1 or epoch % 5 == 0:

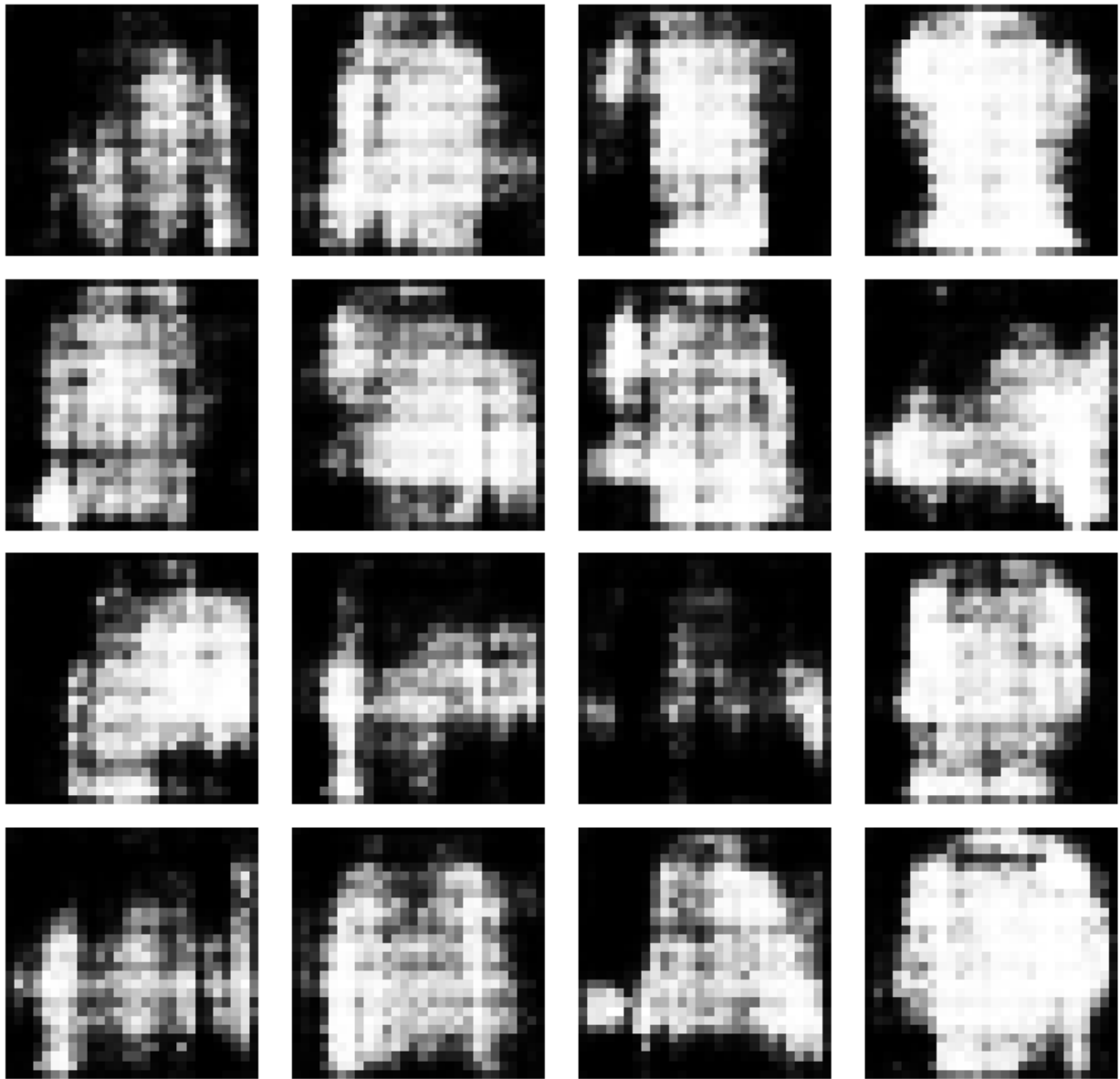
        generate_and_plot_images(
            generator,
            epoch
        )

```

Starting GAN Training...

Epoch 01/30 | Generator Loss: 0.7335 | Discriminator Loss: 1.3543

Generated Images - Epoch 1



Epoch 02/30 | Generator Loss: 0.8933 | Discriminator Loss: 1.2317

Part C : Visualise the Generated Images

At regular intervals during training, generate a fixed set of images using the same random latent vectors.

Display the generated images after:

Initial training

Intermediate training

Final training

For example:

Epoch 1 Epoch 10 Epoch 25 Epoch 50

[images] [images] [images] [images]

Question

What changes do you observe as training progresses?

```
# =====  
# 15. FINAL GENERATED IMAGES  
# =====  
  
print("\nFinal Generated Images")  
  
generate_and_plot_images(  
    generator,  
    EPOCHS  
)  
  
# =====  
# 16. PLOT GENERATOR AND DISCRIMINATOR LOSSES  
# =====  
  
plt.figure(figsize=(10, 5))  
  
plt.plot(  
    generator_losses,  
    label="Generator Loss"  
)  
  
plt.plot(  
    discriminator_losses,  
    label="Discriminator Loss"  
)  
  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.title("GAN Training Loss")  
  
plt.legend()  
plt.grid(True)  
  
plt.show()  
  
# =====  
# 17. GENERATE 100 NEW IMAGES  
# =====
```

```

noise = tf.random.normal(
    [100, LATENT_DIM]
)

generated_images = generator(
    noise,
    training=False
)

generated_images = (
    generated_images + 1
) / 2.0

# =====
# 17. GENERATE 100 NEW IMAGES
# =====

noise = tf.random.normal(
    [100, LATENT_DIM]
)

generated_images = generator(
    noise,
    training=False
)

generated_images = (
    generated_images + 1
) / 2.0

# =====
# 18. DISPLAY 100 GENERATED IMAGES
# =====

plt.figure(figsize=(12, 12))
for i in range(100):
    plt.subplot(10, 10, i + 1)

    plt.imshow(
        generated_images[i, :, :, 0],
        cmap="gray"
    )

    plt.axis("off")

plt.suptitle(
    "100 Synthetic Fashion Images",
    fontsize=16
)

```

```

plt.tight_layout()
plt.show()

# =====
# 19. SAVE THE TRAINED GENERATOR
# =====

generator.save(
    "fashion_mnist_gan_generator.keras"
)

print(
    "\nGenerator saved as "
    "fashion_mnist_gan_generator.keras"
)

# =====
# 20. GENERATE ONE NEW IMAGE
# =====

new_noise = tf.random.normal(
    [1, LATENT_DIM]
)

new_image = generator(
    new_noise,
    training=False
)

new_image = (
    new_image + 1
) / 2.0

plt.figure(figsize=(4, 4))

plt.imshow(
    new_image[0, :, :, 0],
    cmap="gray"
)

plt.title("New Synthetic Fashion Image")
plt.axis("off")

plt.show()

```

Part E: Analysis

Answer the following questions based on your results.

Q1. Generator Analysis

Do the generated images resemble Fashion-MNIST images?

Give examples from your generated output.

Q2. Discriminator Analysis

What happens to discriminator accuracy during training?

Does very high discriminator accuracy necessarily mean that the GAN is performing well?

Explain.

Q3. Loss Analysis

Plot:

Generator Loss vs Epoch Discriminator Loss vs Epoch

Interpret the graphs.

Q4. Mode Collapse

Inspect your generated images.

Do many generated images look almost identical?

If yes, explain the phenomenon of mode collapse.

If no, explain what evidence suggests that the Generator is producing diverse samples.

Q5. Training Stability

Identify whether your GAN training appears:

Stable

Unstable

Oscillating

Collapsing

Support your answer using the loss graphs and generated images.

Q6. Quality vs Diversity

Consider these two situations:

Model A

Generates very realistic images but almost all images look similar.

Model B

Generates less realistic images but produces many different types of fashion items.

Which property should be considered when evaluating a GAN?

Explain why image quality alone is insufficient.

Part F: Experimental Investigation

Now perform one controlled experiment.

Change ONE of the following:

Experiment 1

Change the latent vector dimension.

For example:

50 → 100

Compare the results.

OR

Experiment 2

Change the number of training epochs.

For example:

20 → 50

OR

Experiment 3

Change the Generator architecture.

OR

Experiment 4

Change the Discriminator architecture.

OR

Experiment 5

Change the learning rate.

Part G: Experimental Table

Complete:

Parameter Experiment 1 Experiment 2

Latent dimension

Epochs

Learning rate

Generator architecture

Discriminator architecture

Final Generator loss

Final Discriminator loss

Observed image quality

Image diversity

Mode collapse observed?

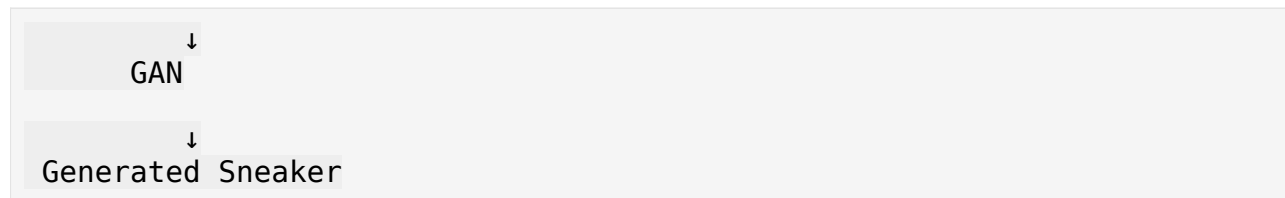
Part H: Think Beyond the Basic GAN

Suppose the fashion company now says:

"We don't want the GAN to generate any random fashion item. We want to specify which category should be generated."

For example:

Input: "Sneaker"



Q1. What modification to a basic GAN would you consider?

Q2. Would a Conditional GAN (cGAN) be suitable? Explain why.

Q3. Design a conceptual architecture for this system.